6.2 Ejercicio de programación 3 y pruebas de unidad

Francisco Natanael Ortiz Martínez

A01796014

Introduction

This assignment presents the design and implementation of a Hotel Reservation System developed in Python using object-oriented principles. The system models three core abstractions Hotel, Customer, and Reservation and implements persistent behavior through structured JSON data storage. By organizing the application into separate layers for models, services, and storage, the project promotes modularity, separation of concerns, and maintainability. The implementation adheres strictly to PEP8 coding standards and passes flake8 and pylint validations, ensuring clean, readable, and professional-grade source code.

A central focus of this project is the development of a comprehensive unit testing suite using Python's unittest framework. Each service method is exercised through carefully designed test cases that validate normal operations, edge conditions, and error scenarios such as invalid JSON input and unavailable rooms. Achieving more than 85% line coverage ensures that the majority of the application logic is verified automatically. The tests not only validate correctness but also safeguard against regressions, making future modifications safer and more predictable.

Unit testing significantly enhances the changeability and long-term sustainability of the system. Because all core behaviors are validated by automated tests, refactoring or extending functionality can be performed with confidence. When changes are introduced, the test suite immediately reveals unintended side effects. Combined with strict adherence to coding standards and static analysis tools, the project demonstrates how disciplined engineering practices lead to robust, maintainable, and production-quality software.

Evidence of executions

main.py 1, U | models.py U | storage.py U | services.py U | test_system.py 2, U | {} hotels.json U | {} customers.json U | {} reservations.json U

Users > fnortiz > Documents > Master > 7.- Jan 2026 > Pruebas de software y aseguramiento de la calidad > Ejercicio de programacion 3 > A01796014_A6.2 > hotel_system > main.py > ...

```python


def main():
    """Run simple demo operations."""

    print("\n--- Hotels ---")
    hotels = HotelService.display_hotels()
    for hotel in hotels:
        print(hotel)

    print("\n--- Customers ---")
    customers = CustomerService.display_customers()
    for customer in customers:
        print(customer)

    print("\n--- Creating New Reservation ---")
    reservation = ReservationService.create_reservation(
        "c1",
        "h2"
    )
    print("Created reservation:", reservation.to_dict())

    print("\n--- Updated Hotels ---")
    hotels = HotelService.display_hotels()
    for hotel in hotels:
        print(hotel)


if __name__ == "__main__":
    main()

```

PROBLEMS 3 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS | SQL HISTORY | TASK MONITOR

```
❯ flake8 .
❯ pylint models.py services.py storage.py tests/test_system.py main.py

------------------------------------
Your code has been rated at 10.00/10

❯ coverage run -m unittest discover tests
coverage report
.....Error reading data/hotels.json: Expecting value: line 1 column 1 (char 0)
....
----------------------------------------------------------------------
Ran 9 tests in 0.009s

OK
Name                     Stmts   Miss  Cover
--------------------------------------------
models.py                   13      0   100%
services.py                 88      0   100%
storage.py                  17      3    82%
tests/test_system.py        63      1    98%
--------------------------------------------
TOTAL                      181      4    98%
❯ python3.14 main.py

--- Hotels ---
{'hotel_id': 'h1', 'name': 'Grand Palace', 'location': 'New York', 'total_rooms': 10, 'available_rooms': 8}
{'hotel_id': 'h2', 'name': 'Ocean View', 'location': 'Miami', 'total_rooms': 5, 'available_rooms': 5}

--- Customers ---
{'customer_id': 'c1', 'name': 'Alice Johnson', 'email': 'alice@example.com'}
{'customer_id': 'c2', 'name': 'Bob Smith', 'email': 'bob@example.com'}

--- Creating New Reservation ---
Created reservation: {'reservation_id': 'dec8a006-c734-4027-9689-73ea1705d84a', 'customer_id': 'c1', 'hotel_id': 'h2'}

--- Updated Hotels ---
{'hotel_id': 'h1', 'name': 'Grand Palace', 'location': 'New York', 'total_rooms': 10, 'available_rooms': 8}
{'hotel_id': 'h2', 'name': 'Ocean View', 'location': 'Miami', 'total_rooms': 5, 'available_rooms': 4}

[1] [] ~/Doc/Ma/7/P/Ejercicio de programacion 3/A01796014_A6.2/hotel_system  on [] [] main ?1
❯
```

Conclusion

The development of this Hotel Reservation System demonstrates the importance of combining clean architecture, coding standards, and automated testing in software engineering. By enforcing PEP8 compliance and achieving clean results in flake8 and pylint, the codebase maintains high readability and consistency. More importantly, the unit tests serve as a safety net that validates both functional requirements and edge cases, ensuring system reliability.

Emphasizing unit testing throughout the assignment significantly improves the system's changeability. Because behavior is verified through automated tests, enhancements or refactoring can be performed without fear of breaking existing functionality. This project illustrates how strong testing practices, measurable code coverage, and adherence to coding standards collectively produce software that is not only correct, but maintainable, extensible, and professionally engineered.